

Real-Time Soccer Analytics Pipeline

A Computer Vision System for Automated Tactical and Physical Match Analysis from Broadcast Footage

Ashwin Ramaseshan

August 16, 2026

Table of Contents

Abstract

Modern football analysis increasingly depends on spatial and physical data: how far players run, how fast they move, how much space they control, and how territorial dominance shifts across a match. This information is traditionally locked behind expensive commercial optical tracking systems that require multiple fixed cameras installed in a stadium, placing it out of reach for the majority of clubs, academies, and independent analysts. This report presents a complete computer vision pipeline that reconstructs this spatial layer from ordinary single-camera broadcast footage, with no manual annotation at run time. The system is founded on a custom-trained keypoint detection model that locates 48 distinct pitch reference points in every frame; these points are matched to their known real-world coordinates on a standard 105 m × 68 m pitch and used to estimate a per-frame homography that maps the moving broadcast image onto true pitch coordinates. On this calibrated foundation, together with object detection, jersey-based team classification, tracking, identity linking, and camera-motion estimation, six analytical modules are built: player tracking with speed and distance, player stamina monitoring, per-player space control via Voronoi tessellation, team-level pitch control, camera-compensated off-ball movement trails, and a curated tactical events system that surfaces the most significant movement-based moments in a match. This report documents the system in full, including the mathematics of the homography, the coordinate conventions, the design and automated labeling of the 48 reference points, the diagnosis and correction of jersey-color contamination in team assignment, the diagnosis and mitigation of tracking identity fragmentation, the redesign of the events system away from unreliable ball detection and toward a ranked and curated presentation, and the separate optimized pipeline built to extend processing from short clips to full-length matches. It closes with a new account of the system's validation at scale across 59 clips spanning seven broadcast sources – including a fragmentation bug specific to goalkeeper identities, a latent nondeterminism bug in team-color clustering, and a quantified profiling study of where full-match processing time actually goes – and a candid account of the system's remaining limitations and the directions that follow from them.

1 Introduction

Football is fundamentally a game of space and movement. The most consequential moments in a match often occur away from the ball: a defender stepping into a passing lane, a striker drifting into an unmarked pocket, a full-back sprinting to recover during a counter-attack. Yet the statistics that dominate mainstream analysis – goals, assists, passes, tackles – capture almost none of this. They record discrete on-ball events while ignoring the continuous spatial context in which those events occur. As the former defender Paolo Maldini observed, “if I have to make a tackle then I have already made a mistake”: the finest defensive work is positional and preventative, and therefore invisible to conventional event data.

Elite clubs address this gap with optical tracking systems that record the position of every player many times per second. These systems produce the spatial data needed to quantify pressing intensity, defensive line height, territorial control, and off-ball movement. But they are expensive, they typically require several calibrated cameras permanently installed in the stadium, and they are inaccessible to the vast majority of the football world. For anyone working from ordinary broadcast footage – a single camera that constantly pans, tilts, and zooms – this rich spatial layer has historically been unavailable.

1.1 Objective

The objective of this project was to build a fully autonomous pipeline that closes this gap: a system that accepts standard broadcast footage and produces structured tactical and physical analytics, without any manual tagging at run time. Rather than depending on specialized multi-camera installations, the system works from the same footage any viewer sees on television, making detailed spatial analysis possible for any match with available video – including opposition players and transfer targets who could never be asked to wear a club’s own tracking hardware.

Delivering this required solving a chain of interlocking computer vision problems. Players, referees, and the ball must be detected and tracked over time. Players must be assigned to their correct teams despite similar colors, shadows, and motion blur. The moving broadcast camera must be calibrated so that positions measured in the image can be translated into true positions on the pitch. Tracking identities, which fragment whenever players are occluded or leave the frame, must be stabilized. Only once these foundations are in place can meaningful physical and spatial metrics be computed reliably.

1.2 Contributions and Structure

This report describes a working system that addresses these challenges and produces five distinct analytical outputs. Its principal contributions are: a custom-trained pitch keypoint model that detects 48 reference points per frame to enable automatic per-frame calibration; a jersey-color team classification method made robust to grass and background contamination; a player identity-linking system that maintains consistent identities through occlusions and camera cuts; five integrated analysis modules operating on this shared calibrated foundation; and, new in this revision, a systematic validation of the entire pipeline against 59 held-out video segments drawn from seven distinct broadcast sources, which surfaced and closed two

previously undetected bugs and produced a quantified, evidence-based understanding of where the system’s remaining computational cost actually lies.

Section 2 documents the full system architecture and the engineering behind every foundational component, including the homography mathematics, the coordinate conventions, the reference-point design and automated labeling, the team-classification fix, camera-motion estimation, and the identity-fragmentation diagnosis. Sections 3–7 describe each of the first five analytical modules in detail, together with their value to coaches and scouts. Section 8 documents the sixth module, tactical events, including its redesign away from unreliable ball detection and the diagnostic process that produced its curated, ranked presentation. Section 9 describes the separate optimized pipeline built to extend the system from short clips to full-length matches. Section 10 documents the pipeline’s validation at scale: the batch validation methodology, the generalization results across held-out broadcast sources, the diagnosis and fix of a goalkeeper-specific identity-fragmentation bug, a latent nondeterminism bug found in team-color clustering, and a quantified profiling study of full-match processing cost. Section 11 discusses limitations and future work, and Section 12 concludes.

2 System Architecture

The pipeline is organized as a sequence of stages, each building on the output of the previous. Raw broadcast video enters at one end; five annotated analytical videos emerge at the other. This section walks through that pipeline in full detail.

2.1 Pipeline Overview

At a high level, processing proceeds in five foundational stages followed by five rendering modules. First, an object detection and tracking stage locates every player, goalkeeper, referee, and the ball in each frame and assigns each a persistent tracking identity. Second, a team classification stage sorts the detected players into two teams from jersey color. Third – and most critically – a calibration stage detects the 48 pitch reference points and computes a homography for each frame, establishing the mapping between image and pitch coordinates. Fourth, a camera-motion estimation stage measures how the broadcast camera moves between frames. Fifth, an identity-linking stage stabilizes player identities across occlusions. The five rendering modules then consume this shared foundation to produce their respective visualizations.

A central design principle is that the expensive foundational stages – tracking, calibration, and camera-motion estimation – are computed once and cached to disk. Because their results are deterministic for a given input video, they are serialized after the first run and reloaded instantly thereafter. During development this separation was essential: refining a visualization did not require repeating the multi-minute calibration pass each time, which would otherwise have made iteration prohibitively slow.

2.2 Object Detection and Tracking

The detection stage uses a fine-tuned object detection model trained on the four relevant classes in football footage: players, goalkeepers, referees, and the ball. For each frame the model

returns bounding boxes for every detected object. A tracking algorithm then links these per-frame detections across time, assigning each object a stable identity so that a given player retains the same identity number as they move about the pitch. For each player, the point of interest for all downstream analysis is the bottom-centre of their bounding box – the position of their feet on the pitch surface – since it is this ground-contact point, not the centre of the body, that corresponds to a true location on the two-dimensional plane of the field.

Tracking from broadcast footage is inherently imperfect. Players occlude one another, cluster at set pieces, leave and re-enter the frame as the camera pans, and are periodically lost to motion blur. This gives rise to identity fragmentation, addressed in Section 2.6.

2.3 Team Classification and the Contamination Problem

Assigning each player to their correct team is a prerequisite for nearly all tactical analysis. The method clusters players by jersey color: the colors of all detected players are grouped into clusters, and the two largest clusters are taken to be the two teams, with the smallest absorbing referees and outliers.

The naive implementation of this idea failed, and the manner of its failure is instructive. The first version sampled the average color of the upper third of each player's bounding box. At broadcast scale a player occupies only a small region of the frame, so this loose crop included a substantial amount of background grass. The contamination pulled every player's sampled color toward a similar muted green. When the two team clusters were measured, their colors were found to be separated by only a small margin in color space – so small that the confidence threshold for assignment exceeded the separation itself. In concrete terms, a clearly red-jerseyed player was being sampled as a muddy green and assigned to the opposing green team, because the red signal had been almost entirely washed out by grass.

The fix was to sample far more precisely. Rather than the upper third, the system samples a tight torso region: the central band of the bounding box horizontally, avoiding the left and right edges where background bleeds in, and the chest area vertically, below the head and above the shorts. Within that region, pixels are converted to HSV color space and filtered to remove grass-green hues, very dark shadow pixels, and low-saturation background pixels, retaining only genuine jersey-colored pixels. The team color is computed from the mean of the surviving pixels, and the clustering is aggregated over several frames rather than a single one for robustness. This change increased the measured separation between the two team colors by roughly a factor of four, moving it well clear of the assignment threshold, and eliminated the misclassification: the previously mislabeled red player was now sampled as clearly red and assigned correctly. Once a player is confidently assigned, the result is cached; genuinely ambiguous players – typically those tightly clustered with others – are left unassigned rather than guessed, and are resolved on a later frame when their jersey is more cleanly visible.

2.4 Pitch Calibration and the Homography

The heart of the system is pitch calibration, the component that makes real-world spatial analysis possible at all. The problem is one of homography estimation: recovering the projective transformation that relates the two-dimensional camera image to the flat plane of the pitch.

2.4.1 Mathematical formulation

Because the pitch is planar, the relationship between a point on the pitch and its image under a pinhole camera is a homography – a projective transformation represented by a 3×3 matrix H acting on homogeneous coordinates. Writing a pitch point as (x, y) in metres and its image as (u, v) in pixels, the mapping from pixels to pitch is

$$[x', y', w]^T = H \cdot [u, v, 1]^T, x = x' / w, y = y' / w. \quad (1)$$

The final division by the homogeneous coordinate w is essential: it is what encodes perspective. Omitting it – a subtle but consequential error – causes projected points to skew badly rather than lying flat on the pitch, and was in fact the cause of an early failure in the spatial-overlay modules before the normalization was made explicit. The inverse transformation H^{-1} maps pitch coordinates back to pixels and is used whenever a quantity computed in pitch space, such as a zone boundary, must be drawn back onto the video frame.

A homography has eight degrees of freedom and is determined by four or more point correspondences between image and pitch, provided no three are collinear. In practice many more correspondences are used, and the transformation is estimated robustly so that a minority of inaccurate detections do not corrupt the result.

2.4.2 Coordinate conventions

A single, consistent world coordinate system is used throughout. The pitch is represented as a rectangle of 105 m in length by 68 m in width – the standard dimensions of a professional field. The origin is placed at one corner, with the x -axis running along the length of the pitch from 0 to 105 and the y -axis across its width from 0 to 68. Every reference point has a fixed, known location in this system: the centre spot at (52.5, 34), the penalty spots at their regulation distances, the penalty-box and six-yard-box corners at their known offsets, and so on. Establishing this fixed table of real-world coordinates is what allows any detected image point to be assigned a true metric position. A validation check exploits these bounds directly: any computed position falling outside $[0, 105] \times [0, 68]$ is flagged as the product of an unreliable homography for that frame and is discarded before it can corrupt a downstream metric.

2.4.3 The 48 reference points

To detect the pitch markings automatically, a custom keypoint detection model was trained to locate 48 distinct reference points in the image. These points are the natural intersections and landmarks of the pitch markings: the four corners; the intersections of the penalty boxes and six-yard boxes with the goal lines and with each other; the penalty spots and the centre spot; the points where the centre circle meets the halfway line; and additional landmarks around the penalty arcs and centre circle. Together they provide a dense set of correspondences distributed across the pitch, so that whatever portion of the field a given broadcast frame happens to show, several reference points are likely to be visible within it.

2.4.4 Automated labeling of the reference points

Training this model required a labeled dataset in which each frame is annotated with the pixel locations of its visible reference points. Because a broadcast camera rarely shows the whole

pitch, only the subset of the 48 points actually within a given frame is labeled; points outside the frame are marked as not visible rather than guessed. Each training image is thus associated with a label recording, for all 48 points, either a pixel location and a visibility flag or an indication of absence.

An important subtlety emerged in how invisible points are represented. The annotations export placeholder coordinates for invisible points rather than true nulls, so the visibility flag, not the coordinate value, must be treated as authoritative; a point marked invisible must be excluded from both the training loss and the calibration regardless of the placeholder coordinate attached to it. Recognizing this prevented invisible points from silently contaminating the model. To translate the trained model's raw index-numbered detections into the fixed world-coordinate table, detected keypoints were visualized with their index labels overlaid on sample frames, and each index was mapped to its corresponding pitch landmark and thereby to its known real-world coordinate.

2.4.5 Training and per-frame estimation

The model was trained to predict the pixel location of each visible reference point. An early training run of too few epochs at low resolution produced a revealing failure mode: the model learned to localize the pitch region – detecting that a pitch was present and roughly where – but could not accurately place the individual keypoints within it, because keypoint localization for many spread-out points requires far more optimization than coarse region detection. Extending training substantially and increasing the input resolution resolved this, yielding a model that reliably detects the visible reference points across a wide range of camera angles. Higher resolution mattered because the reference points are fine, precisely located features that are easily lost when the image is downsampled.

For each frame, the detected reference points are matched to their known world coordinates and a homography is estimated robustly, with inaccurate correspondences rejected automatically. A further safeguard removes duplicate correspondences: if two distinct detected points are mapped to the same world coordinate, the resulting system is degenerate and produces a wildly incorrect transformation, so such duplicates are filtered before estimation. Because the camera moves continuously, the homography is recomputed for every frame rather than fixed once, so that panning, tilting, and zooming are all accounted for. Where too few reliable reference points are visible in a frame to support a trustworthy homography, the system falls back to the most recent good calibration rather than producing a bad one.

2.4.6 Manual correction of the reference-point mapping

A separate and distinct source of error emerged after the keypoint model itself was performing well: the fixed table mapping each of the 48 keypoint indices to its real-world coordinate was, for several indices, simply wrong, having been assigned by visual inspection of a small number of sample frames rather than verified systematically. This is a different failure from an inaccurate detection – the model could be correctly locating a given pitch marking in the image, while the table incorrectly asserted that marking corresponded to the wrong real-world location, guaranteeing an incorrect homography regardless of detection quality. A telling symptom of this class of error was two keypoint indices being assigned identical world

coordinates despite corresponding to visibly different pixel locations, which renders the correspondence system mathematically degenerate.

To correct this systematically rather than by further guesswork, a small interactive labeling tool was built: a detected frame is displayed alongside a scaled diagram of a regulation pitch, and each detected keypoint can be selected and matched, by eye, to its true location on the pitch diagram, with the corrected mapping written back into the reference table. This tool converted an error-prone process of inferring correspondences from static images into a direct, verifiable one, and was used to correct the reference table for the indices found to be wrong. It illustrates a broader lesson from this project: a highly accurate detection model does not by itself guarantee a correct system, since the interpretation layer built on top of its output – here, a lookup table – is just as capable of introducing systematic error, and is often less visible than a detection failure because the model appears, superficially, to be working correctly.

2.4.7 An honest note on calibration accuracy

A candid assessment of the calibration is warranted, because it shaped the design of the later modules. A frame-level success rate – the fraction of frames for which some homography could be computed – is a misleading measure of quality, because a homography fitted only to reference points clustered in one region of the image can extrapolate poorly to other regions. A more stringent test, checking whether every tracked player’s position maps into the physical bounds of the pitch, revealed that per-frame accuracy varies considerably: near-perfect on frames where reference points are well distributed, but degraded on frames where the visible markings are confined to a small area. This finding directly motivated the validation-and-fallback logic used throughout, and informed the decision, in some spatial modules, to favor stability over strict metric fidelity. As documented in Section 10, subsequent large-scale validation confirmed this is best understood as a camera-condition-dependent property rather than a uniform defect: calibration reliability tracks the diversity of visible pitch markings in a given broadcast’s typical framing, holding up strongly on static, wide tactical-camera feeds and degrading measurably on footage with frequent zooms, pans, and close-ups. It remains an important target for future improvement, as discussed in Section 11.

2.5 Camera-Motion Estimation

Several outputs – most importantly the movement trails – must distinguish genuine player motion from apparent motion caused by the camera. When the camera pans to follow play, stationary objects appear to slide across the frame. To separate these effects, the system estimates the camera’s displacement between consecutive frames by tracking stable background features from one frame to the next. This yields a per-frame camera displacement in pixels, which can be subtracted from raw image positions to recover pitch-relative motion. The estimator occasionally produces a large single-frame spike; because such spikes would otherwise corrupt any trail passing through them, the camera-motion signal is smoothed with a median filter before use, which removes isolated spikes without blurring genuine sustained pans.

2.6 Player Identity Linking

The tracker's tendency to assign a new identity to a re-detected player – identity fragmentation – means that a single real player can be represented by several short-lived identities over a match. Diagnosis confirmed the scale of the problem: on a representative clip, roughly forty distinct tracking identities were produced for a scene containing about twenty-two real players, with the surplus arising from players who were briefly lost and then re-acquired under a new number.

The remedy is an identity-linking module. When the tracker loses an identity and shortly afterward produces a new one, the linker recognizes them as the same player on the basis of spatial and temporal proximity: a newly appearing identity that emerges close to where a recently lost identity disappeared, within a short time window and on the same team, is merged with the earlier one. The merged identity inherits the earlier one's history, so that a player's stamina, assigned color, and movement trail persist across the switch rather than resetting. Applied to the representative clip, this linking merged a number of fragments back into their originating players, reducing the count of distinct identities meaningfully toward the true figure. Position-based linking cannot recover every case – a player who travels far during a long occlusion, or reappears after the matching window has closed, cannot be confidently re-associated – and closing this residual gap is identified in Section 11 as a target for appearance-based re-identification. A companion mechanism guards against the opposite failure: an identity that has not been seen for a sustained period has its stale history expired, so that abandoned trails do not linger on the pitch.

3 Module 1: Player Tracking, Speed, and Distance

3.1 How It Works

The first module builds on the calibrated foundation to produce the most fundamental physical metrics: each player's speed and cumulative distance covered. Each player's feet position is converted, via the per-frame homography, from image pixels into real-world metres. The distance a player travels between two consecutive frames is the metric distance between their positions in those frames, and their speed is that distance divided by the inter-frame time, converted to kilometres per hour.

Raw frame-to-frame speed is far too noisy to display directly. Small inaccuracies in the bounding box, minor homography errors, and the discrete nature of pixel positions all introduce jitter, so that a stationary player may appear to fluctuate by several kilometres per hour between frames. Two safeguards address this. First, each computed position is validated against the pitch bounds described in Section 2.4; a frame whose homography places a player outside the physical field is treated as momentarily unreliable, and no speed sample is taken from it – the previously displayed value is held rather than allowing a spike or a blank. Second, the displayed speed is a short rolling average over the most recent valid frames rather than an instantaneous value. This smoothing removes jitter while remaining responsive enough to capture genuine acceleration: a burst from a standing start to a sprint registers within a fraction of a second. A physiological ceiling is also imposed, above the fastest recorded human

sprint, so that any computed speed exceeding it is rejected as an artifact rather than displayed. Distance is accumulated only from valid samples, and a jump larger than a player could physically cover in one frame resets the accumulation rather than adding a spurious contribution.

3.2 Why It Is Useful

Speed and distance are the bedrock of physical performance analysis. Total distance covered, and the share of it covered at high intensity, are primary indicators of work rate and physical condition. Coaches use them to monitor fatigue, to compare a player against their own historical baseline, and to judge when a tiring player should be substituted.

For scouts, physical output provides an objective complement to subjective judgment. A player who consistently produces high-intensity running – repeated sprints, rapid recoveries, sustained output over ninety minutes – demonstrates a physical profile directly relevant to whether they can adapt to a more demanding level. Crucially, because this module derives its measurements from broadcast footage rather than a worn device, it makes such profiling possible for any player in any televised match, including opposition players and transfer targets from whom no club could obtain wearable data.

4 Module 2: Player Stamina Monitoring

4.1 How It Works

The stamina module presents a live dashboard, displayed beside the footage, showing an estimated stamina level for every player. Stamina is modeled as a reserve that depletes with movement intensity: sprinting drains it quickly, running and jogging more slowly, and standing or walking almost not at all. Each player begins at full stamina, and their reserve is decremented over time according to the speed measured by Module 1. To keep the display readable, values are refreshed periodically rather than fluctuating on every frame.

The module's defining feature is persistent memory. Football tracking is discontinuous: players constantly leave the frame as the camera follows the ball, and are frequently lost and re-acquired by the tracker. A naive display would reset a player to full stamina each time they were lost, rendering the metric meaningless. To prevent this, the module maintains a persistent record of every player it has ever seen. When a player leaves the frame their stamina is preserved and their entry is marked as temporarily out of view rather than discarded; when they reappear, their history resumes. This persistence is reinforced by the identity-linking system of Section 2.6, so that a player's stamina survives even a change in their underlying tracking identity: when an old identity is superseded by a new one, the panel transfers the accumulated history and displays the current identity going forward.

A further refinement handles players who are off-screen for extended periods, whose movement cannot be observed while they are out of view. The module makes a reasonable assumption about their activity during the gap: a player who re-enters the frame at high speed – as during a counter-attack – has their off-screen period treated as high-intensity effort and

their stamina adjusted accordingly, whereas a player who re-enters walking is assumed to have been moving gently.

4.2 Why It Is Useful

Fatigue is among the most consequential and least visible factors in a match. As players tire, their positioning slips, their decisions degrade, and their injury risk rises. A live view of which players are fading gives a coaching staff a powerful input for in-game decisions: whom to substitute, when to modulate intensity, and which areas of the pitch may become vulnerable as key players tire.

The persistent-memory design is what makes the module practical rather than a novelty. Real footage never offers a clean, continuous view of all twenty-two players at once, so any stamina system that could not survive occlusions and camera cuts would be unusable. By maintaining each player's fitness profile across the whole match regardless of whether they are currently on screen, the module produces a coherent, match-long picture of the entire squad's physical output from a single broadcast feed.

5 Module 3: Player Space Control

5.1 How It Works

The third module visualizes a quantity conventional statistics ignore: how much of the pitch each individual player controls. It uses a Voronoi tessellation – a partition of the plane in which every point is assigned to the nearest player. Each player is thereby surrounded by a region of all locations they are best placed to reach first, a natural proxy for the space under their control.

Each player is given a distinct, stable color, and their region is shaded accordingly, so that the division of territory among all players is legible at a glance. Producing this stably from real footage required solving several problems that a naive implementation does not anticipate. A Voronoi diagram computed over only the players visible in a narrow band of the frame produces regions that stretch unboundedly toward the edges; the tessellation is therefore bounded by guard points placed well outside the field and clipped to the visible pitch area, so that regions do not spill into the crowd. The partition is also sensitive: because every point belongs to its nearest player, the entire diagram can restructure when the set of players changes even slightly, which manifests as flickering. This was addressed by recomputing the tessellation at a measured cadence rather than every frame, smoothly interpolating between successive computations, holding the last valid state through brief moments when too few players are cleanly detected, and tolerating short tracking dropouts so that a momentarily lost player does not cause their region to vanish and reappear. Distinct per-player coloring, rather than mere team coloring, was achieved by assigning each merged identity a stable hue, spaced so that neighboring players are easily told apart; goalkeepers are deliberately included, since in a one-on-one the space a goalkeeper commands is precisely the question of interest.

5.2 Why It Is Useful

Space, more than possession or passing, is the true currency of football. The attacking team's central aim is to create and exploit space; the defending team's is to deny it. A Voronoi view makes this invisible contest visible. It reveals which defenders are covering dangerous central areas and which are being pulled out of position; which attackers have found unmarked pockets and which are being crowded out; and how the balance of territory shifts in the critical seconds of a transition or a press.

For a coach, this is a direct window into positional quality – the very attribute conventional defensive statistics fail to capture. A defender who consistently commands the most dangerous zones, without ever needing to make a tackle, exhibits exactly the positional intelligence Maldini's remark celebrates. For a scout, seeing how much space a player habitually controls, and where, offers insight into game intelligence that no count of tackles or interceptions could provide.

6 Module 4: Team Pitch Control

6.1 How It Works

Where Module 3 partitions the pitch among individual players, the fourth module aggregates to the level of the two teams, producing a territorial heatmap of the entire field. Conceptually, every location is colored according to which team has the nearer player and therefore controls that zone. The computation samples a dense grid of points across the pitch and, for each, determines the nearest player of each team and assigns the point to the closer; the result is blended into a soft, continuous heatmap overlaid on the footage, one color washing across the areas each team dominates and the two blending where control is contested. The overlay is confined to the pitch area so that it does not extend into the stands, and it redraws continuously as the teams advance, retreat, and reshape.

6.2 Why It Is Useful

Team pitch control turns an abstract intuition – “this team is dominating territory” – into a concrete, moment-to-moment visualization, and exposes patterns that box-score statistics cannot. A high defensive line reveals itself as control pushed deep into the opponent's half; a low, compact block appears as territory ceded but tightly held in one's own third. Pressing traps become visible as sudden expansions of control into the area where the ball is about to be won, and the speed with which control swings after a turnover reveals how well organized a team is in transition.

For coaching staff, this gives an immediate read on whether the team is executing its intended territorial strategy and how the opponent is responding. For analysts preparing for a fixture, studying an opponent's pitch-control patterns across previous matches reveals structural tendencies – where they concentrate, where they can be stretched, how they behave on losing the ball – reframing analysis away from discrete events and toward the continuous spatial story of the match.

7 Module 5: Off-Ball Movement Trails

7.1 How It Works

The final module traces each player's recent movement, drawing a trail behind them that shows the path taken over a rolling window of the last few seconds. The trail fades from faint at its oldest point to solid at the player's current position, conveying both where a player has been and the direction and pace of their movement.

The central technical challenge is that the broadcast camera is itself in motion. Without correction, a player standing still while the camera pans would leave a long trail composed purely of camera movement. To produce pitch-relative trails, the module uses the smoothed camera-motion estimates of Section 2.5 to compensate each historical position, anchoring the trail to the pitch rather than to the frame; each stored point records the frame in which it was captured so that the correct cumulative camera offset can be applied when it is drawn. Keeping the trails clean required careful handling of the same camera spikes that motivated the median filtering: a lifecycle issue was diagnosed in which, once a candidate point was rejected for jumping too far, the reference against which subsequent points were compared was never updated, so that the trail could lock up and cease growing for the remainder of the match. Separating the always-updating reference position from the filtered trail buffer resolved this, so that an isolated spike is skipped while normal recording resumes immediately on the next frame. Trails are further protected by rejecting outlier points that deviate sharply from their neighbors, and by drawing with anti-aliasing for a smooth line.

7.2 Why It Is Useful

Off-ball movement is among the most important and least documented aspects of football. The run that drags a defender out of position, the recovery sprint of a full-back, the subtle repositioning of a midfielder to offer a passing angle – these frequently decide matches, yet they leave no trace in conventional event data, which records only what happens on the ball. Movement trails make these actions legible.

By visualizing the paths players take, the module lets a coach study the quality and timing of off-ball runs: whether attackers are stretching the defense, whether defenders are holding their shape as they move, and how players coordinate as a unit. For player development, it offers concrete visual feedback on movement patterns that are otherwise hard to describe. And because the trails are pitch-anchored and camera-compensated, they present movement as it truly occurred on the field, independent of how the broadcast happened to film it.

8 Module 6: Tactical Events

8.1 From Ball Events to Movement Events

The original design for this module attempted to detect discrete match events directly from the ball – passes, interceptions, shots, tackles, and dribbles inferred from the ball's proximity to players and its speed and trajectory. In practice this approach was fundamentally limited by the reliability of ball detection itself. The ball in broadcast footage is small, fast-moving, and

frequently occluded by players; a rule such as “the ball moved rapidly from near one player to near a teammate” depends on ball positions that are themselves noisy, and any event logic built on top of that noise inherits it. Attempting to make the rules more sophisticated did not help, because the problem was not the rules but the reliability of their inputs.

This motivated a deliberate pivot: rather than continuing to chase ball-dependent events, the module was redesigned around player position and movement, quantities the pipeline already computes reliably from the calibrated tracking data – world-space position, smoothed speed, camera-compensated movement, and per-player space control. Ten events were defined on this foundation: Sprint (sustained top-speed effort), Burst (sharp acceleration), Press (multiple teammates converging on an opponent), Recovery (a high-speed run toward one’s own goal), Overlap (a wide player running past a teammate down the flank), Space (a player commanding an unusually large region of the pitch), Switch (a long lateral run changing the point of attack), Drop (dropping deep to receive), Break (a fast forward run beyond the defensive line), and Isolated (a wide player cut off from support). None of these require the ball to be detected at all, which removed the single largest source of unreliability in the original design.

8.2 Diagnosing and Correcting Over-Sensitivity

The first working version of the detector produced 977 logged events over a 31-second clip – an implausible rate of roughly thirty-one events per second. The natural first hypothesis was that a cooldown mechanism, intended to prevent the same event from being logged repeatedly for one player, was not functioning. A full audit of every consecutive pair of logged events across all players and event types disproved this: every gap between logged events for a given player and event type met or exceeded the cooldown threshold, with zero violations. The cooldown was working exactly as designed.

The real cause was that the detection thresholds were simply too permissive. A rule such as “speed increases by more than a fixed amount within half a second” is easy for ordinary running variation to cross, not just genuine explosive accelerations; because the cooldown only limits how often a player can re-trigger a given event, an easily-crossed threshold with a short cooldown becomes a near-continuous firing rate rather than a signal of rare, notable moments. This was compounded by a design gap in how continuous conditions were evaluated: sustained-effort events such as Sprint used a rising-edge trigger, firing once when a run began and not again until it ended, whereas events like Burst and Drop re-evaluated a raw condition every frame, so a single sustained spike could be logged as several separate, cooldown-legal “episodes.”

The remedy was threefold. First, every event was converted to a rising-edge trigger, firing once at the onset of a qualifying condition and not again until the condition had cleared. Second, thresholds were raised substantially and, for the most volatile events, a minimum sustained-duration requirement was added so that a single noisy frame could not qualify on its own. Third, a general noise guard was applied: any player whose world-space position failed the pitch-bounds validation on a given frame was excluded from event detection for that frame, since an invalid position – the same artifact discussed in Section 2.4 – is a common source of a phantom, physically impossible speed or displacement spike. These three changes reduced the event count from 977 to a range consistent with genuinely notable moments, with the largest

remaining contributors (events with no inherent duration requirement) tightened further until their behavior matched the sustained-effort events.

8.3 A Curated, Ranked Presentation

Even with realistic firing rates, a continuously scrolling log of individual events is a poor interface: it either shows too little context at once or overwhelms the viewer with a constant stream. The module was therefore redesigned around a windowed, ranked, curated presentation. Events are grouped into fixed twenty-second windows, and within each window every event is scored by a combination of a base importance weight for its type and a normalized intensity score reflecting how far it exceeded its threshold, so that an extreme instance of a lower-priority event type (a player isolated by an exceptional distance, an exceptionally fast sprint) can outrank a routine instance of a higher-priority type. The base weights were deliberately chosen to reflect tactical significance rather than raw frequency: events that signal danger, opportunity, or a defining tactical moment (a forward run beyond the line, a dangerous player isolated in space) rank above routine physical actions that occur constantly (acceleration, dropping to receive).

To keep the displayed batch varied rather than dominated by a single high-scoring event type, a diversity cap limits any one event type to at most three appearances in a window's top ten, with the highest-scoring instances of that type retained. The resulting batch – up to ten events, fewer if a quiet passage of play produced fewer qualifying moments, none at all if none did – is displayed as a continuously cycling carousel for the duration of its twenty-second window before being replaced by the next window's batch. Two of the ten event definitions, Isolated and Overlap, were further refined to require that the players involved be positioned in a wide channel of the pitch, since both are meaningful specifically as wide-play phenomena – a winger cut off from support, or a full-back overlapping a winger down the flank – and were otherwise firing on central players for whom the underlying geometric pattern held without the tactical meaning being present.

8.4 An Honest Assessment of Accuracy

It is worth being direct about what this module does and does not achieve. These are threshold-based detectors operating on positional and kinematic data, not a model trained on labeled examples of real tactical events; they will not match the judgment of a human analyst reviewing the same footage. The physically-grounded events (Sprint, Recovery) are the most reliable, since they measure quantities the pipeline already validates carefully. The more interpretive events (Press, Overlap, Switch) infer tactical intent from geometry and are correspondingly more prone to occasional mislabeling. The module is best understood, and is presented, as a system for surfacing tactically interesting signal from continuous tracking data rather than as a claim of precise event classification; closing this gap with a trained model is discussed in Section 11.

9 Scaling to Full-Length Matches

The modules above were developed and validated primarily on short clips of roughly thirty seconds, for which the full pipeline – calibrating every frame at full resolution – completes in a

manageable time. Extrapolated naively to a full match of ninety minutes, this every-frame, full-resolution approach becomes impractical on ordinary consumer hardware without a dedicated GPU: the calibration stage alone, which invokes the keypoint model on every single frame, dominates the runtime, and a full match at that rate would require on the order of a full day of processing.

Two observations motivated a second, explicitly separate processing pipeline built for longer footage, developed alongside – and without modifying – the original, validated implementation. First, the broadcast camera’s position changes little in the fraction of a second between adjacent frames, so recomputing the full homography on every single frame is unnecessary; sampling the keypoint model on a small fraction of frames and interpolating the homography matrix between them preserves accuracy to a degree indistinguishable in practice, while cutting the dominant cost by close to an order of magnitude. Second, the same reasoning applies to player detection: a player’s position changes smoothly enough between consecutive frames that running the full detection model on every third frame and linearly interpolating bounding boxes for the frames in between introduces negligible error under normal play, with the caveat that a player newly appearing in frame is not interpolated until a real detection confirms them. Reducing the resolution at which frames are fed to the detection and keypoint models for inference, then rescaling the resulting coordinates back to full resolution, provided a further reduction in cost, at a small and generally imperceptible cost to detection precision for distant or fast-moving objects.

A separate, practical constraint emerged when applying this pipeline to a full match: the existing rendering functions, by design, operate on the entire video held in memory as a sequence of frames, an approach that is entirely reasonable for short clips but that would require an impractical amount of memory for a full match of tens of thousands of frames. Rather than rewrite this proven rendering logic, the long-video pipeline separates the one-time, memory-safe foundational stage – which streams through the video frame by frame and never holds more than the current frame in memory – from the rendering stage, which loads and renders a bounded window of the cached foundation data at a time. This preserves the exact, validated rendering behavior of the original modules while making full-length processing tractable: the expensive foundational computation is paid once for the whole match, after which any window of interest can be rendered quickly from the cache.

Applying this pipeline to a full-length match confirmed both the practicality and the honest limits of a CPU-only setup: total foundational processing for a match of approximately ninety minutes completed in several hours rather than the day or more the naive approach would have required, and the resulting player tracking, team assignment, and event detection all held up qualitatively on footage from an entirely different stadium, broadcast, and pair of teams than the clips used during development – a meaningful test of generalization, since nothing in the pipeline was tuned to this specific footage in advance. This confirms that, while a dedicated GPU remains the clear path to processing full matches quickly, the system does not require one to produce a genuine end-to-end result. Section 10 extends this single-clip generalization check into a systematic, multi-source validation study and a precise, profiled account of where the remaining processing time goes.

10 Validation at Scale

The modules described in Sections 2–9 were developed and refined against a small number of clips, primarily the original tuning footage and the single additional clip used in Section 9’s generalization check. That is a reasonable basis for building a system, but it is a weak basis for trusting one: a pipeline tuned against one or two videos can silently encode assumptions – about kit colors, camera framing, broadcast production style, or frame rate – that happen to hold for that footage without holding more broadly. This section documents a deliberate effort to test that assumption directly, at scale, and reports what it found: two real bugs, previously undetected, and a precise, evidence-based account of where the pipeline’s remaining computational cost actually lies.

10.1 A Batch Validation Harness

To move beyond spot-checking individual clips, a batch validation harness was built to run the full pipeline against many video segments automatically and score each one against fixed, objective thresholds – team-classification resolution rate and cluster separation, calibration confidence (the fraction of tracked players whose transformed position lands inside the physical pitch rectangle), camera-motion stability, goalkeeper-track count, ball-detection rate, and identity-fragmentation indicators – producing a per-clip scorecard and an overall pass, degraded, or fail verdict.

Two engineering problems in the harness itself were found and fixed before its results could be trusted. First, an early version of the harness took orders of magnitude longer per clip than the pipeline’s own reported per-frame cost implied; profiling traced this to the harness’s frame-access pattern, which repeatedly re-seeked into the compressed source video rather than reading the needed window once, sequentially, into memory – an access-pattern bug entirely separate from the pipeline being validated, and unrelated to any of its actual computational cost. Reading each clip’s frames once, sequentially, brought harness overhead down by roughly two orders of magnitude with bit-identical scoring output, confirming the fix changed only performance and not correctness. Second, once genuine per-clip cost was visible, a further discovery – detailed in Section 10.4 – traced part of the harness’s own instability to an unseeded source of randomness inside the pipeline itself, not the harness.

10.2 Test Set and Methodology

Fifty-nine thirty-to-sixty-second segments were sampled from five video sources not used during development of any pipeline component, spanning seven distinct leagues and broadcast productions and chosen specifically to probe conditions the original tuning footage did not cover, including a green home kit – a genuine stress test of the grass-contamination fix described in Section 2.3, since a green jersey and green turf are adversarial to precisely the color-based separation that fix relies on – and a broadcast at a different native frame rate. Segments were partitioned by source video, not by individual segment, into a development set of thirty-nine segments used to investigate and fix any issues found, and a holdout set of twenty segments – including the green-kit source – withheld until after development-set fixes were finalized, so that the holdout result would measure genuine generalization rather than iterative tuning against the same data.

Within the sampling process itself, segments were required to be continuous live match play: candidate windows were rejected if they fell within a pre-match, half-time, or post-match portion of the source video, or if automated checks detected a broadcast scene cut or a shot that was not the main tactical camera angle (a close-up or replay), ensuring every scored segment represented the kind of continuous broadcast footage the pipeline is actually designed to process.

10.3 Generalization Results

Across the development set, team-classification resolution and calibration confidence were broadly consistent with the levels observed on the original tuning footage, with the pattern already anticipated in Section 2.4.7: calibration confidence correlated with the diversity of pitch markings visible in a source's typical camera framing rather than with the source itself, holding up strongly on a static, wide tactical-camera feed and degrading measurably on sources with more dynamic broadcast direction – frequent pans, zooms, and cuts to closer shots. This was confirmed directly by inspecting individual frames from the weaker-performing windows, which showed the keypoint model correctly detecting the pitch markings actually visible in frame, with the degraded confidence arising from those markings being confined to a small region rather than from any detection error.

On the holdout set – entirely unseen during development, and including the green-kit source specifically targeting the grass-contamination fix – team-classification and calibration results fell within a comparable range to the development set, with team-classification resolution on the holdout set in fact slightly stronger on average. This is the central, positive result of the validation effort: performance on genuinely unseen broadcast sources was not meaningfully worse than on sources used during development, indicating that the fixes described throughout this report – the HSV-based jersey sampling, the fallback-and-reliability logic in calibration, the identity-linking system – generalize as intended rather than having been implicitly overfit to the footage used to design them.

10.4 Goalkeeper Identity Fragmentation

The batch validation surfaced a specific, previously undiagnosed instance of the identity-fragmentation problem described in Section 2.6, affecting goalkeepers rather than outfield players. Three segments, drawn from three different source videos, failed the goalkeeper-count check despite otherwise strong team-classification and calibration scores; in each case, visual inspection of the flagged tracking identities confirmed that a single real goalkeeper had been split across five to nine separate tracking identities over the course of the segment, rather than several distinct goalkeepers genuinely being present.

The root cause was a structural gap in the existing identity-linking module: it had been built and validated exclusively against outfield-player fragments and explicitly excluded goalkeeper identities from merge consideration, on the historical assumption that goalkeepers, who move within a small area near their own goal, were unlikely to fragment in the same way outfield players do. The validation data disproved this assumption directly – goalkeepers fragment for the same underlying tracking reasons as any other player, most visibly during goalmouth congestion, when a goalkeeper is repeatedly and briefly occluded by defenders and attackers contesting a cross or a corner.

The fix extended the existing, already-validated fragment-merging logic to goalkeeper identities on the same spatial-and-temporal-proximity basis already used for outfield players, rather than introducing a new, goalkeeper-specific heuristic. All three flagged segments were reprocessed and passed the goalkeeper-count check after the fix, and a full regression pass across all fifty-nine validation segments confirmed no outfield-player merging behavior was altered by the change.

10.5 A Latent Nondeterminism Bug in Team Classification

A full-baseline regression check performed after the goalkeeper fix – rerunning all fifty-nine previously scored segments and comparing every result against its prior scorecard, to confirm the fix introduced no unintended side effects – surfaced an unrelated, previously undetected issue: one segment’s team-classification result differed meaningfully between the original run and the regression run, despite both runs processing bit-identical input data.

The cause was traced to the underlying color-clustering routine’s random initialization, which was left unseeded. Under ordinary conditions this is harmless, because the two team-color clusters are typically separated widely enough in color space that the clustering converges to the same result regardless of its random starting point. The single affected segment, however, had jersey colors whose three-way population split – the two team clusters plus the referee-and-outlier cluster the routine also fits – was close enough to tied that different random initializations occasionally converged to a different, incorrect grouping of which cluster represented which team. A systematic scan of all fifty-nine segments’ cluster population balances confirmed this was a rare, data-dependent condition rather than a pervasive one: only one further segment showed a comparably close population split, and had – by chance – not yet been observed to flip between runs, meaning it carried the same latent risk without having manifested it.

The fix fixes the random seed used by the clustering call immediately before each invocation, eliminating the run-to-run variability entirely; five repeated runs on the previously unstable segment after the fix produced bit-for-bit identical output every time. This finding is reported candidly because of what it illustrates about validating a system of this kind: a bug that is invisible on the overwhelming majority of test cases, and that manifests only as an occasional, unexplained discrepancy on a rerun of otherwise-identical data, is exactly the class of defect that ad hoc spot-checking is least likely to catch, and that a systematic, repeated, full-baseline regression process is well suited to surface.

10.6 A Quantified Profiling Study of Full-Match Processing Cost

Section 9 established that full-match processing is practical on CPU-only hardware but reported its cost only in aggregate. A subsequent, dedicated profiling study instrumented every stage of the pipeline to determine precisely where that time is spent, motivated by the observation that the pipeline’s actual runtime on some segments substantially exceeded expectations set by earlier, more favorable footage.

The finding was unambiguous: on footage where the base ball-detection model’s primary recall is weak, a secondary, higher-accuracy fallback detector – a tiled inference pass invoked specifically on frames where the primary detector fails, using a finer grid than an earlier,

incorrect assumption about its configuration had suggested – dominates total processing time overwhelmingly, measured at up to roughly three hundred times the combined cost of every other pipeline stage together on a segment where the primary detector missed the ball on approximately eighty percent of frames. This is not a defect in the fallback detector’s design; the detector’s whole purpose is to trade substantial per-frame cost for meaningfully higher accuracy precisely where the fast, primary model fails, and the resulting rate is fixed by the accuracy the fallback model itself provides.

Two software-level mitigations were investigated directly rather than assumed. Multi-threaded parallelization of the fallback detector’s internal tiled inference calls was tested at several worker-count settings and produced no clean, reproducible speedup under the conditions measured, with results confounded by system-level resource contention during testing rather than yielding a clear negative result either way – an honest limitation of the experiment rather than a settled conclusion, and one identified as worth revisiting under more controlled conditions. True batched inference, in which multiple image tiles are submitted to the underlying model in a single forward pass rather than sequentially, was investigated and ruled out definitively: inspection of the model’s own configuration confirmed that batched inference is not supported by the underlying model architecture, closing off this avenue rather than leaving it open to further, fruitless engineering effort.

This profiling study sharpens, rather than changes, the conclusion already reached in Section 9 and reiterated in Section 11: dedicated GPU acceleration is the clear remaining lever for full-match processing speed, and this section replaces a general expectation of that fact with a specific, measured account of exactly which stage would benefit, and by roughly how much a given hardware improvement could be expected to help.

10.7 GPU Acceleration of the Fallback Detector: Confirmation and Measurement

[New in this revision.] Section 10.6 identified the fallback ball-detection stage as the pipeline’s dominant cost on demanding footage and concluded, on profiling evidence alone, that GPU acceleration was the clear remaining lever. This section reports the first attempt to actually deploy that stage on a GPU, the debugging required to get it running correctly, and the measured result — confirming the earlier analytical conclusion with hardware data.

Deployment environment. The fallback detector was deployed to a rented cloud GPU instance running the pipeline’s existing, unmodified codebase, with the goal of exercising the exact same fallback-triggering code path already profiled in Section 10.6, unchanged, on GPU hardware instead of CPU.

A misleading initial failure mode. Installing the fallback detector’s dependencies on the GPU instance initially failed with a cascading sequence of unrelated-looking import errors, each pointing at a different, unrelated large model class inside the detector library’s own eager model-registration code — the kind of failure that invites chasing each error individually as if it were a separate bug. Tracing the full stack trace rather than patching each surface symptom in turn revealed a single, shared root cause: a version mismatch between the installed deep-learning framework and its audio-processing companion library, triggered incidentally by an unrelated import path deep inside the detector library’s own startup sequence. Once the two libraries were reinstalled against matching versions, the entire chain of seemingly unrelated

failures disappeared at once. This is a specific, concrete instance of a pattern already emphasized elsewhere in this report (Sections 2.4.6 and 12): a failure that presents as many small, unrelated problems is frequently one problem observed from many different vantage points, and is best diagnosed by tracing root cause rather than patching each symptom.

Two further, smaller configuration issues were found and fixed before the fallback detector ran correctly on GPU: the model-inference runtime had been installed in its CPU-only build rather than its GPU-enabled build, and a required low-level GPU-interop dependency was missing entirely. Both were resolved with standard package reinstalls once identified; neither required any change to the detector's own code or trained weights.

Measured result. With the fallback detector correctly running on GPU, its per-trigger processing cost was measured directly against the same CPU-bound cost profiled in Section 10.6, on the same class of high-trigger-rate footage. Per-trigger cost fell from a CPU-bound range of roughly 13.7–14.3 seconds per trigger to roughly 0.20–0.24 seconds per trigger on GPU — a measured 65–70× speedup on precisely the stage Section 10.6 identified as dominating total cost by up to 300×. On a 30-second test clip with a near-worst-case trigger rate (the fallback detector fired on 496 of 502 frames), the complete pipeline — all stages, unchanged — finished end to end in approximately six and a half minutes with GPU acceleration enabled, in a regime where the CPU-only cost model from Section 10.6 would very plausibly place the same run in the multi-hour range.

What this does and does not establish. This result confirms, with real measured hardware data, the conclusion Section 10.6 reached by profiling and reasoning alone: the fallback detector is not merely the theoretical bottleneck but the practical one, and GPU acceleration addresses it directly and substantially, with no accuracy trade-off, since the detector's own weights and logic were unchanged. It does not yet establish full-match processing under any particular time budget: the other five rendering modules documented in Sections 3–7 are, to varying degrees, CPU/OpenCV-bound rather than GPU-bound, and at least one of them (the pitch-control heatmap render of Section 6) was separately observed to run at a fairly fixed frame rate regardless of GPU tier, since its cost lies in per-pixel grid computation rather than model inference. Getting a full 90-minute match processed end to end in, for instance, under thirty minutes would therefore require either a substantially higher tier of GPU than was tested here, genuine distributed processing across multiple GPUs, or further optimization of the CPU-bound rendering stages specifically — a materially larger engineering effort than accelerating a single inference stage, and one not yet undertaken. This GPU deployment is best understood as confirming that the single largest, best-identified cost in the pipeline responds exactly as predicted to the fix already proposed for it in Section 10.6 — not as a claim that full-match processing is now fast in an absolute sense.

11 Limitations and Future Work

The system is a functional demonstration, and its current limitations point directly to the most valuable directions for future development.

Calibration accuracy. As documented in Section 2.4 and confirmed at scale in Section 10.3, the calibration is not uniformly accurate: because the keypoint model can only observe the

markings visible in a given frame, the resulting homography is most reliable where those markings appear and can extrapolate poorly to areas of the pitch with few visible reference points, and this effect is now known to correlate specifically with how dynamic a given broadcast's camera direction is. The most impactful improvement would be to strengthen this calibration – by training the keypoint model on substantially more data spanning a wider variety of camera angles and stadiums, or by adopting a calibration approach that also exploits the pitch's line and circle geometry rather than point landmarks alone. Because every downstream module rests on the homography, better calibration would improve all of them at once, and in particular would allow the space-control and pitch-control visualizations to be rendered as undistorted top-down views in true metric coordinates.

Tracking identity. Although the identity-linking of Section 2.6, and its extension to goalkeepers in Section 10.3, reduces fragmentation, position-based matching cannot recover every case, and some players remain represented by more than one identity across a full match. Incorporating appearance-based re-identification – recognizing a player by jersey number or visual features rather than position alone – would close much of this remaining gap and further stabilize every per-player metric.

Event recognition. The tactical events module of Section 8 demonstrates that meaningful signal can be extracted from movement data alone using threshold-based rules, but a natural and high-value next step is to recognize genuine on-ball events – passes, interceptions, shots, tackles, dribbles – ideally through a dedicated model trained on labeled examples rather than heuristic rules. This would first require a substantially more reliable ball-tracking model than broadcast footage currently permits with the present detector, since, as discussed in Section 8, ball-dependent rules were abandoned specifically because ball detection was not reliable enough to build on. Combined with the existing spatial layer, reliable event detection would allow every event to be analyzed in its full spatial context, uniting the “what happened” of traditional data with the “where and how” this pipeline provides.

Interactive tactical tools. The pipeline currently produces finished analytical videos. A compelling direction is an interactive tool allowing a coach to pause on any frame, select a player, and reposition them – turning the system from a passive analysis engine into an active coaching instrument for demonstrating where players should have been.

Beyond a single camera. While reliance on single-camera broadcast footage is precisely what makes the system accessible, fusing multiple angles where available would improve both tracking continuity and calibration accuracy for clubs able to capture their own footage.

Dedicated compute. Section 9 demonstrates that a complete match can be processed without a dedicated GPU, and Section 10.6's profiling study now quantifies precisely where that runtime goes: on demanding footage, the fallback ball-detection pass alone can account for on the order of three hundred times the cost of every other stage combined, making it overwhelmingly the dominant cost of full-match processing rather than one contributor among several. Software-level mitigations for this specific stage were investigated directly and found limited – true batched inference is not supported by the underlying model at all, and multi-threaded parallelization did not yield a clear, reproducible improvement under the conditions tested. This sharpens rather than weakens the case for dedicated compute: because the cost is concentrated so heavily in a single, well-identified, GPU-amenable stage, access to even modest

GPU acceleration is the most straightforward and highest-leverage remaining lever for making full-match processing practical at scale, with a far more direct and predictable payoff than further CPU-side software optimization of that stage is likely to provide.

12 Conclusion

This report has documented an end-to-end computer vision pipeline that converts ordinary broadcast football footage into structured tactical and physical analytics, with no manual annotation at run time. At its foundation is a custom-trained model that detects 48 pitch reference points in every frame; matched to their known real-world coordinates on a 105 m × 68 m pitch, these points drive a per-frame homography that translates the moving broadcast image into real-world positions. Around this core sit the supporting systems that make the analysis reliable in practice: a team-classification method hardened against grass contamination, a camera-motion estimator smoothed against spikes, and an identity-linking system that holds players' identities together through the occlusions and camera cuts that pervade real footage. Upon this foundation, six modules deliver complementary views of a match – physical output through speed and distance, squad fitness through stamina, individual spatial dominance through Voronoi space control, territorial balance through pitch-control heatmaps, off-ball movement through camera-compensated trails, and a curated tactical events system that surfaces the most tactically significant moments from continuous movement data.

Beyond the individual modules, this project is also a case study in the discipline that reliable computer vision systems demand: nearly every capability described here reached its working form only after a wrong initial assumption was diagnosed and corrected – a homography silently producing coordinates outside the pitch, a team classifier contaminated by grass, a reference table with two keypoints assigned the same real-world location, an event detector whose apparent cooldown bug turned out to be a threshold problem, a rendering pipeline that could not fit a full match into memory. In each case, the fix followed only from measuring the system's actual behavior against ground truth rather than assuming the design was correct. The separate optimized pipeline built to extend the system to full-length matches, without altering the original validated implementation, reflects the same underlying principle: proven, working components were preserved and extended rather than rewritten, and a claim of readiness was made only after being tested against footage genuinely different from anything used during development.

This same discipline, applied at a larger scale in Section 10, is what distinguishes this revision from the system's initial validation. A batch validation harness scoring fifty-nine held-out segments across seven distinct broadcast sources did not simply confirm the system worked; it found a real, previously undiagnosed bug in how goalkeeper identities were tracked, a latent and rare nondeterminism bug in team-color classification that had been invisible to every prior, single-run test, and a precise, quantified answer to the question of where the system's remaining processing cost actually lies – replacing a general sense that CPU-only processing was slow with a specific, measured account of exactly one stage's disproportionate cost and exactly which software-level fixes for it had been tried and had, honestly, not worked. Each of these findings would very plausibly have gone undetected without a systematic process built specifically to surface exactly this class of rare, data-dependent, or condition-specific failure.

Taken together, these components demonstrate that spatial intelligence once available only through expensive multi-camera stadium installations can, to a meaningful degree, be recovered from the same footage any viewer sees on television. The system is not without limitations – calibration accuracy and tracking identity in particular remain open challenges, honestly acknowledged – but it establishes a working foundation, a validated claim of generalization across genuinely unseen broadcast sources, and a clear, evidence-based path forward. In making space, movement, and physical output visible from broadcast video alone, it points toward a future in which rich tactical analysis is accessible not only to the wealthiest clubs, but to any coach, scout, or analyst with a match to study.